

UNIT 3: PYTHON COMPLEX DATA TYPES

Introduction to Strings:

- In Python, a **string** is a sequence of characters enclosed within **quotes**.
- Strings are widely used to store text data such as names, messages, or any combination of letters, digits, and symbols.

How to Create Strings in Python

We can create strings in Python using three types of quotation marks:

1. Single Quotes

```
a = 'Hello'
print(a)
```

Output:

```
Hello
```

2. Double Quotes

```
b = "Python"
print(b)
```

Output:

```
Python
```

3. Triple Quotes (used for multi-line strings)

```
c = """This is
a multi-line
string."""
print(c)
```

Output:

```
This is  
a multi-line  
string.
```

Empty String

- An **empty string** is a string that has no characters.
- It is written as just a pair of quotes:

```
empty = ""  
print(empty)
```

Output:

Nothing is printed because the string is empty.

Important Properties of Strings

1. Strings are Immutable

- Once created, the characters of a string **cannot be changed**.
- Example:

```
name = "Python"  
# name[0] = 'J' # ✗ Error, strings cannot be modified
```

2. Indexing in Strings

- Each character in a string has a **position (index)**.
- **Forward Indexing** → starts from 0 to length-1
- **Backward Indexing** → starts from -1 to -length

Example:

```
word = "HELLO"
print(word[0]) # H (first character)
print(word[4]) # O (last character using forward index)
print(word[-1]) # O (last character using backward index)
print(word[-5]) # H (first character using backward index)
```

Output:

```
H
O
O
H
```

Summary

- Strings are sequences of characters enclosed in 'single', "double", or """triple""" quotes.
- They can be empty ("").
- Strings are **immutable** (cannot be changed after creation).
- Characters in a string are accessed using **indexes** (both positive and negative).

Traversing a String in Python:

What does Traversing Mean?

- **Traversing** a string means **visiting each character of the string one by one**.
- Since a string is a **sequence of characters**, we can use its **indexes** or a **loop** to access each character.

Ways to Traverse a String

1. Using a for loop (direct character access)

- The simplest way to traverse a string is by using a for loop.
- In this method, the loop automatically picks each character from the string one by one.

Example:

```
name = "superb"

for ch in name:
    print(ch, "-", end=" ")
```

Output:

```
s - u - p - e - r - b -
```

2. Using Indexing with for loop

- We can also traverse a string using indexes.
- This is useful when we want both the index and the character.

Example:

```
word = "Python"

for i in range(len(word)):
    print("Index:", i, "Character:", word[i])
```

Output:

```
Index: 0 Character: P
Index: 1 Character: y
Index: 2 Character: t
Index: 3 Character: h
Index: 4 Character: o
Index: 5 Character: n
```

3. Using a while loop

- We can also use a while loop to traverse a string.

Example:

```
text = "Hello"
i = 0

while i < len(text):
    print(text[i], end=" ")
    i += 1
```

Output:

```
H e l l o
```

Summary

- Traversing means going through each character of the string.
- We can traverse a string:
 1. Directly using a for loop
 2. Using indexes in a for loop
 3. Using a while loop
- This is very useful for string processing tasks like counting vowels, searching letters, or reversing a string.

Example:

1. Print each character of a string

```
# Program to print each character of a string
string1 = input("Enter a string: ")

print("The characters of the string are:")
for ch in string1:
    print(ch, end=" ")
```

Output:

```
Input: Hello
Output: H e l l o
```

2. Count the number of vowels

```
# Program to count vowels in a string
string1 = input("Enter a string: ")

count = 0
vowels = "aeiouAEIOU"
```

```
for ch in string1:
    if ch in vowels:
        count += 1

print("Number of vowels =", count)
```

Output:

Input: superb

Output: Number of vowels = 2

3. Print characters with their index

```
# Program to print characters with their index
string1 = input("Enter a string: ")

print("Characters with their indexes are:")
for i in range(len(string1)):
    print("Index", i, "→", string1[i])
```

Output:

Input: Python

Output:

Index 0 → P

Index 1 → y

Index 2 → t

Index 3 → h

Index 4 → o

Index 5 → n

String Operators in Python

Introduction:

In Python, we can use several operators with strings to manipulate them in different ways.

Some of the commonly used operators are:

1. **Concatenation (+)**
2. **Replication (*)**
3. **Membership Operators (in, not in)**
4. **Comparison Operators (==, !=, <, >, <=, >=)**

Since **strings are immutable**, whenever we apply these operators, **a new string is created** instead of modifying the original one.

1.Concatenation Operator (+)

- The + operator joins (concatenates) two strings together.

Example:

```
str1 = "Hello"  
str2 = "World"  
  
result = str1 + " " + str2  
print(result)
```

Output:

```
Hello World
```

2. Replication Operator (*)

- The * operator repeats a string multiple times.

Example:

```
word = "Hi "  
result = word * 3
```

```
print(result)
```

Output:

```
Hi Hi Hi
```

3.Membership Operators (in, not in)

- in → checks if a substring exists inside a string.
- not in → checks if a substring does not exist.

Example:

```
text = "Python Programming"

print("Python" in text)    # True
print("Java" in text)     # False
print("C++" not in text)  # True
```

Output:

```
True
False
True
```

4.Comparison Operators

- Strings can be compared using relational operators (==, !=, <, >, <=, >=).
- Comparisons are done based on **lexicographical order** (like dictionary order, using ASCII/Unicode values).

Example:

```
print("apple" == "apple") # True
print("apple" != "banana") # True
```

```
print("cat" < "dog")    # True (because 'c' comes before 'd')  
print("Zebra" > "apple") # False (because 'Z' has smaller ASCII value  
than 'a')
```

Output:

```
True  
True  
True  
False
```

Summary

- + → Concatenates strings
- * → Repeats string
- in / not in → Check for substring presence
- Relational operators → Compare strings alphabetically

Checking ASCII Values in Python:

1.What is ASCII?

- Every character in Python (letters, digits, symbols) has a unique **ASCII / Unicode value**.
- For example:
- 'A' → 65
- 'a' → 97
- '0' → 48

2. Function to Check ASCII Values

In Python, we use the **ord()** function to find the ASCII value of a character.

Syntax:

```
ord(character)
```

Example Program

```
# Program to check ASCII values of characters
print("ASCII value of 'A' is:", ord('A'))
print("ASCII value of 'a' is:", ord('a'))
print("ASCII value of 'Z' is:", ord('Z'))
print("ASCII value of 'z' is:", ord('z'))
print("ASCII value of '0' is:", ord('0'))
print("ASCII value of '9' is:", ord('9'))
```

Output

```
ASCII value of 'A' is: 65
ASCII value of 'a' is: 97
ASCII value of 'Z' is: 90
ASCII value of 'z' is: 122
ASCII value of '0' is: 48
ASCII value of '9' is: 57
```

3. Why is this useful?

When we compare strings in Python (like "A" < "a"), Python is actually comparing their **ASCII values**.

Example: Comparing with ASCII

```
print("Is 'A' < 'a'? :", 'A' < 'a') # True (65 < 97)
print("Is 'Z' < 'a'? :", 'Z' < 'a') # True (90 < 97)
print("Is '9' > '1'? :", '9' > '1') # True (57 > 49)
```

Output

```
Is 'A' < 'a'? : True  
Is 'Z' < 'a'? : True  
Is '9' > '1'? : True
```

4. Reverse: Getting Character from ASCII

We can also use the **chr()** function to convert an ASCII value back to a character.

```
print(chr(65)) # A  
print(chr(97)) # a  
print(chr(48)) # 0
```

Output:

```
A  
a  
0
```

Summary:

- Use `ord(char)` → gives ASCII value.
- Use `chr(number)` → gives character for that ASCII.
- String comparison in Python is based on these ASCII values.

String Slices in Python:

Definition:

A **string slice** is a **substring** (part of a string) that is extracted by specifying a **range of indices**.

Python allows you to “slice” a string to get the portion you need, instead of using the whole string.

General Syntax:

```
string_variable[start : end]
```

- **start** → The index (position) where the slice begins. (Included)
- **end** → The index (position) where the slice ends. (Excluded)
- The result contains characters from **start index up to (end-1)**.
- If you skip start, it begins from the start of the string.
- If you skip **end**, it continues till the **end of the string**.

Indexing in Python Strings

Every character in a string has a position number called **index**.

For the string:

```
word = "amazing"
```

- **Positive indices:**

```
a m a z i n g
0 1 2 3 4 5 6
```

- **Negative indices (counting from the end):**

```
a m a z i n g
-7 -6 -5 -4 -3 -2 -1
```

Examples of String Slices

```
word = "amazing"
```

1. Full String

```
print(word[0:7]) # Output: amazing
```

Explanation: Starts from index 0 to index 6 (7 is excluded).

2. Slice First 3 Characters

```
print(word[0:3]) # Output: ama
```

Explanation: Takes characters at positions 0, 1, and 2.

3. Slice Using Negative Indices

```
print(word[-5:-1]) # Output: azin
```

Explanation:

String Functions and Methods in Python:

Introduction:

- In Python, **strings** are sequences of characters enclosed in single (' '), double (" "), or triple (''' ' or "" "" """) quotes.
- Every string object in Python is actually an **instance of the str class**.
- Python provides many **built-in methods** for string manipulation.
- General Syntax:

```
<string_object>.<method_name>(parameters)
```

1. capitalize() Method:

Definition

The `capitalize()` method in Python is a **string method** that returns a copy of the string with its **first character converted to uppercase** and the **rest of the characters converted to lowercase**.

Syntax

```
string.capitalize()
```

Example

```
txt = "python is FUN!"
```

```
x = txt.capitalize()
print(x)
```

Output

```
Python is fun!
```

2. count() Method:

Definition

- Returns the number of times a specified substring occurs in the string.

Syntax

```
string.count(value, start, end)
```

Example

```
txt = "I love apples, apple are my favorite fruit"
x = txt.count("apple", 10, 24)
print(x)
```

Output

```
1
```

3. endswith() Method:

Definition

- Checks if a string ends with a specified substring.
- Returns **True** if it does, otherwise **False**.

Syntax

```
string.endswith(value, start, end)
```

Example

```
txt = "Hello, welcome to my world."  
x = txt.endswith("my world.")  
print(x)
```

Output

```
True
```

4. find() Method:

Definition

- Finds the **index of the first occurrence** of a specified substring.
- Returns -1 if the value is not found.
- Similar to index(), but index() raises an error if not found.

Syntax

```
string.find(value, start, end)
```

Example

```
txt = "Hello, welcome to my world."  
x = txt.find("e")  
print(x)
```

Output

```
1
```

5. replace() Method:

Definition

- Replaces all occurrences of a substring with another substring.

Syntax

```
string.replace(oldvalue, newvalue, count)
```

Example

```
txt = "one one was a race horse, two two was one too."  
x = txt.replace("one", "three")  
print(x)
```

Output

```
three three was a race horse, two two was three too.
```

6. split() Method:

Definition

- Splits a string into a **list** of substrings based on a given separator.
- Default separator is **whitespace**.

Syntax

```
string.split(separator, maxsplit)
```

Example

```
txt = "hello, my name is Peter, I am 26 years old"
x = txt.split(", ")
print(x)
```

Output

```
['hello', 'my name is Peter', 'I am 26 years old']
```

7. join() Method:

Definition

- Joins all elements of an iterable (list, tuple, dictionary, etc.) into a single string.
- A string must be provided as the **separator**.

Syntax

```
string.join(iterable)
```

Example

```
myDict = {"name": "John", "country": "Norway"}
mySeparator = "TEST"
x = mySeparator.join(myDict)
print(x)
```

Output

```
nameTESTcountry
```

8. isalpha() Method:

Definition

- Returns **True** if all characters in the string are **alphabetic (a–z or A–Z)**.
- Returns **False** if the string contains numbers, spaces, or special characters.

Syntax

```
string.isalpha()
```

Example

```
txt = "Company10"  
x = txt.isalpha()  
print(x)
```

Output

```
False
```

9. isalnum() Method:

Definition

- Returns **True** if all characters are **alphanumeric** (letters and numbers).
- Returns **False** if the string contains spaces or special characters.

Syntax

```
string.isalnum()
```

Example

```
txt = "Company 12"  
x = txt.isalnum()  
print(x)
```

Output

```
False
```

Lists in Python:

Definition

- A **list** in Python is a **collection of ordered elements** (items) that can store values of **any type** (integers, strings, floats, even other lists).
- Lists are enclosed in **square brackets []** with elements separated by commas.
- Lists are **mutable**, meaning their elements can be **changed, added, or removed** after creation.

Key Features of Lists:

1. **Ordered** – Elements have a defined order and can be accessed using indices
2. **Mutable** – Elements can be modified without creating a new list.
3. **Heterogeneous** – A list can store multiple data types (e.g., integers, strings, floats together).
4. **Supports Nesting** – Lists can contain other lists (multi-dimensional lists).

Examples of Lists:

```
[ ]          # Empty list
[1, 2, 3]     # List of integers
[1, 2.5, 4.5, 4] # List of numbers (integers + floats)
["a", "b", "c"] # List of strings
[1, "a", 2.5, "zero"] # List of mixed value types
```

Creating and Accessing Lists in Python:

1. Creating Lists:

- To create a list, put elements inside square brackets [], separated by commas.

```
my_list = [10, 20, 30, 40]
```

2. Nested Lists:

- A list that contains another list as an element is called a **nested list**.

```
L1 = [3, 4, [5, 6], 7]
```

- Here:
 1. L1 has 4 elements: 3, 4, [5, 6], and 7.
 2. L1[2] → [5, 6] (a list itself).
 3. Length of L1 = 4 (since [5, 6] is counted as one element).

3. Accessing List Elements:

- Similar to strings, list elements are accessed using indexes:

```
vowels = ['a', 'e', 'i', 'o', 'u']

print(vowels[0]) # 'a'
print(vowels[2]) # 'i'
print(vowels[1:4]) # ['e', 'i', 'o'] (slicing)
```

4. Traversing a List:

- **Traversal** means accessing and processing each element of the list.
- Done using **loops**:

```
L = ['P', 'y', 't', 'h', 'o', 'n']

for a in L:
    print(a)
```

Output:

```
P
y
t
h
o
n
```

List Operations in Python:

1. Joining Lists

- We can join (concatenate) two lists using the + operator.
- Both operands **must be lists**.

```
list1 = [1, 2, 3, 4, 5]
list2 = [6, 7, 8]

result = list1 + list2
print(result)
```

Output:

```
[1, 2, 3, 4, 5, 6, 7, 8]
```

Note: You cannot add a number or other value directly to a list using +.

2. Repeating or Replicating Lists

- Use the * operator to repeat a list a given number of times.

```
list1 = [1, 2, 3, 4]

result = list1 * 2
print(result)
```

Output:

```
[1, 2, 3, 4, 1, 2, 3, 4]
```

3. Slicing Lists

- We can extract a **portion (slice)** of a list using indexes.
- Syntax:

```
seq = L[start:stop]
```

- Rules:
 - Starts from index start.
 - Stops **before** index stop.
 - The result is a new list (a slice).

Example:

```
List1 = [10, 20, 30, 40, 50, 60]
```

```
seq = List1[2:-1]
```

```
print(seq)
```

Output:

```
[30, 40, 50]
```

4. Working with Lists

We can perform various operations on lists such as:

- **Appending** (adding elements at the end).
- **Updating** (changing elements by index).
- **Deleting** (removing elements).

Examples:

```
L = [10, 20, 30]
```

```
L.append(40)    # Append element
```

```
print(L)        # [10, 20, 30, 40]
```

```
L[1] = 25      # Update element
print(L)      # [10, 25, 30, 40]

del L[2]      # Delete element at index 2
print(L)      # [10, 25, 40]
```

Appending Elements to a List

- We can add new items to an existing list using the **append()** method.
- `append()` always adds the element at the **end of the list**.
- Syntax:

```
list_name.append(item)
```

Example:

```
lst1 = [10, 12, 14]

lst1.append(16) # Adding a single element at the end

print(lst1)
```

Output:

```
[10, 12, 14, 16]
```

Key Point: `append()` adds **only one element at a time**.

Updating and Deleting Elements in a List:

1. Updating Elements

- To update (change) an element in a list, simply assign a **new value** to the desired index.
- Syntax:

```
List[index] = new_value
```

Example:

```
lst1 = [10, 12, 14, 16]

lst1[2] = 24 # Updating the element at index 2

print(lst1)
```

Output:

```
[10, 12, 24, 16]
```

2. Deleting Elements

(a) Using del Statement

- We can remove an element at a given index, or a slice of elements.
- Syntax:

```
del List[index]    # Removes element at index
del List[start:stop] # Removes elements in range
```

Example:

```
lst = [10, 12, 14, 16]
```

```
del lst[2] # Removes element at index 2
print(lst) # [10, 12, 16]
```

- If we use del list_name, it deletes the **entire list object**.

```
del lst
```

```
# Now lst does not exist anymore
```

(b) Using pop() Method

- pop() removes an element **and returns it**.
- Syntax:

```
List.pop(index) # Removes element at given index  
List.pop()      # Removes last element (default)
```

Example:

```
lst = [10, 12, 14, 16]  
  
print(lst.pop()) # Removes and returns last element → 16  
print(lst)       # [10, 12, 14]  
  
print(lst.pop(1)) # Removes and returns element at index 1 → 12  
print(lst)        # [10, 14]
```

Difference between del and pop():

- del only deletes the element.
- pop() deletes **and returns** the removed element (useful if we want to store it).

List Functions and Methods in Python:

- Built-in functions and methods allow powerful **list manipulation**.
- General syntax:

```
list_object.method(arguments)
```

1. index() Method

- Returns the position of the **first occurrence** of a specified value.

```
fruits = [4, 55, 64, 32, 16, 32]
x = fruits.index(32)
print(x)
```

Output:

```
3
```

2. append() Method

- Adds a **single element** to the end of the list.

```
a = ["apple", "banana", "cherry"]
b = ["Ford", "BMW", "Volvo"]

a.append(b)
print(a)
```

Output:

```
['apple', 'banana', 'cherry', ['Ford', 'BMW', 'Volvo']]
```

3. extend() Method

- Adds all elements of another **iterable** (list, tuple, etc.) to the list.

```
fruits = ['apple', 'banana', 'cherry']
points = (1, 4, 5, 9)

fruits.extend(points)
print(fruits)
```

Output:

```
['apple', 'banana', 'cherry', 1, 4, 5, 9]
```

4. insert() Method

- Inserts an element at a **specific position**.

```
fruits = ['apple', 'banana', 'cherry']  
fruits.insert(1, "orange")  
print(fruits)
```

Output:

```
['apple', 'orange', 'banana', 'cherry']
```

5. pop() Method

- Removes the element at the given index (default: last element) and **returns it**.

```
fruits = ['apple', 'banana', 'cherry']  
x = fruits.pop(1)  
print(x)    # Removed element  
print(fruits) # Updated list
```

Output:

```
banana  
['apple', 'cherry']
```

6. remove() Method

- Removes the **first occurrence** of a specified element.

```
fruits = ['apple', 'banana', 'cherry']  
fruits.remove("banana")  
print(fruits)
```

Output:

```
['apple', 'cherry']
```

7. clear() Method

- Removes **all elements** from the list.

```
fruits = ['apple', 'banana', 'cherry', 'orange']  
fruits.clear()  
print(fruits)
```

Output:

```
[]
```

8. count() Method

- Returns the **number of occurrences** of a specified value.

```
fruits = ['apple', 'banana', 'cherry']  
x = fruits.count("cherry")  
print(x)
```

Output:

```
1
```

9. reverse() Method

- Reverses the elements of the list.

```
fruits = ['apple', 'banana', 'cherry']  
fruits.reverse()  
print(fruits)
```

Output:

```
['cherry', 'banana', 'apple']
```

10. sort() Method

- Sorts the list in **ascending order** by default.

- Can use parameters:
 - `reverse=True` → descending order.
 - `key=function` → custom sorting.

```
cars = ['Ford', 'BMW', 'Volvo']  
cars.sort()  
print(cars)
```

Output:

```
['BMW', 'Ford', 'Volvo']
```

Tuples in Python:

1. Introduction

- A **tuple** is a sequence data type in Python used to store multiple values of any type.
- **Tuples are immutable** → once created, their elements cannot be changed.
- **Difference from lists:**
 - **List** → mutable (elements can be changed).
 - **Tuple & String** → immutable (cannot be changed directly).

2. Creating Tuples

- Tuples are created by enclosing elements in **parentheses ()**, separated by commas.

```
my_tuple = (1, 'apple', 3.14) # Tuple with mixed data types  
empty_tuple = ()             # Empty tuple  
single_element_tuple = (5,)  # Tuple with one element (note the comma)
```

Without the comma, (5) would just be treated as an integer, not a tuple.

3. Accessing Tuple Elements

- Elements are accessed using **indexes**, similar to lists.
- Indexing starts at **0**.

```
my_tuple = (1, 'apple', 3.14)

print(my_tuple[0]) # Output: 1
print(my_tuple[1]) # Output: apple
print(my_tuple[2]) # Output: 3.14
```

So, tuples are like lists, but **immutable**. They are often used to store **fixed collections of items**.

Tuple Operations in Python

1. Concatenation (+)

- Tuples can be joined together using the **+** operator.

```
tuple1 = (1, 2)
tuple2 = (3, 4)

result = tuple1 + tuple2
print(result)
```

Output:

```
(1, 2, 3, 4)
```

2. Repetition (*)

- Tuples can be repeated using the ***** operator.

```
my_tuple = ('hello',) * 3  
print(my_tuple)
```

Output:

```
('hello', 'hello', 'hello')
```

3. Slicing

- Tuples support slicing to extract a portion of elements.

```
my_tuple = (1, 2, 3, 4, 5)  
  
print(my_tuple[1:4])
```

Output:

```
(2, 3, 4)
```

These are the three **main tuple operations**: **Concatenation**, **Repetition**, and **Slicing**.

Tuple Functions in Python:

1. cmp() Function (Only in Python 2)

- Used to compare two tuples.
- Returns:
 - 0 → if tuples are equal
 - 1 → if first tuple is greater
 - -1 → if first tuple is smaller

Syntax:

```
cmp(t1, t2)
```

Example (Python 2 only):

```
T1 = (10, 20, 30)
T2 = (100, 200, 300)
T3 = (10, 20, 30)

print(cmp(T1, T2)) # -1
print(cmp(T1, T3)) # 0
print(cmp(T2, T1)) # 1
```

Note: cmp() was **removed in Python 3**.

In Python 3, comparisons use relational operators (==, <, >, etc.).

2. len() Function

- Returns the **number of elements** in a tuple.

```
T2 = (100, 200, 300, 400, 500)
print(len(T2))
```

Output:

```
5
```

3. max() Function

- Returns the **largest element** in a tuple.

```
T = (100, 200, 300, 400, 500)
print(max(T))
```

Output:

```
500
```

4. min() Function

- Returns the **smallest element** in a tuple.

```
T = (100, 200, 300, 400, 500)
print(min(T))
```

Output:

```
100
```

These are the **common tuple functions** used in Python.

Tuple Methods in Python:

Since **tuples are immutable**, they support only **two built-in methods**:

1. count()

- Returns the number of times a specified value appears in the tuple.

Example:

```
my_tuple = (1, 2, 3, 2, 2, 5)
print(my_tuple.count(2))
```

Output:

```
3
```

2. index()

- Searches the tuple for a specified value.
- Returns the **index (position)** of the **first occurrence** of that value.

Example:

```
my_tuple = (1, 2, 3, 2, 2, 5)
print(my_tuple.index(5))
```

Output:

```
5
```

If the value is not found, `ValueError` is raised.

That's it — **only `count()` and `index()`** are available for tuples because they **cannot be modified**.

Dictionaries in Python

1. Introduction

- A **dictionary** stores **key-value pairs**.
- Unlike lists, dictionary keys can be **any data type** (not just integers).
- Dictionaries are **unordered** → items are not stored in any particular sequence.
- Syntax:

```
my_dict = {'key1': 'value1', 'key2': 'value2', ..., 'keyn': 'valuen'}
```

- Example:

```
A = {1: "one", 2: "two", 3: "three"}  
print(A)  
# Output: {1: 'one', 2: 'two', 3: 'three'}
```

2. Creating and Initializing Dictionaries

- Using `dict()` function:

```
D = dict()  
print(D) # Output: {}
```

- Adding items:

```
H = dict()  
H["one"] = "keyboard"  
H["two"] = "Mouse"
```

```
H["three"] = "printer"
H["Four"] = "scanner"
print(H)
# Output: {'Four': 'scanner', 'three': 'printer', 'two': 'Mouse', 'one': 'keyboard'}
```

3. Traversing a Dictionary

- Use a for loop to access keys and values:

```
H = {'Four': 'scanner', 'three': 'printer', 'two': 'Mouse', 'one': 'keyboard'}

for i in H:
    print(i, ":", H[i], end=" ")
```

```
# Output: Four: scanner one: keyboard three: printer two: Mouse
```

4. Working with Dictionaries

4.1 Adding/Appending Values

- Add a new key-value pair:

```
a = {"mon": "monday", "tue": "tuesday", "wed": "wednesday"}
a["thu"] = "thursday"
print(a)
```

```
#Output: {'thu': 'thursday', 'wed': 'wednesday', 'mon': 'monday', 'tue': 'tuesday'}
```

4.2 Merging Dictionaries

- Use update() to merge two dictionaries:

```
d1 = {1:10, 2:20, 3:30}
```

```
d2 = {4:40, 5:50}
d1.update(d2)
print(d1)
```

```
# Output: {1: 10, 2: 20, 3: 30, 4: 40, 5: 50}
```

4.3 Removing Items

- Remove a specific item using del:

```
A = {"mon":"monday","tue":"tuesday","wed":"wednesday","thu":"thursday"}
del A["tue"]
print(A)
```

```
# Output: {'thu': 'thursday', 'wed': 'wednesday', 'mon': 'monday'}
```

5. Dictionary Functions and Methods

5.1 cmp() (Python 2 only)

- Compares two dictionaries.
- Returns:
 - 0 → dictionaries are equal
 - 1 → first dictionary has more items
 - -1 → first dictionary has fewer items

- Syntax: `cmp(d1, d2)`

5.2 len()

- Returns the number of key-value pairs:

```
H = {'Four': 'scanner', 'three': 'printer', 'two': 'Mouse', 'one': 'keyboard'}
print(len(H))
```

```
# Output: 4
```

5.3 clear()

- Removes all items from a dictionary:

```
D = {'mon': 'Monday', 'tue': 'Tuesday', 'wed': 'Wednesday'}  
D.clear()  
print(D)
```

```
# Output: {}
```

5.4 get(key, default=None)

- Returns the value for a key, or default if key doesn't exist:

```
my_dict = {'name': 'Alice', 'age': 25}  
print(my_dict.get('name'))      # Alice  
print(my_dict.get('address', 'N/A')) # N/A
```

5.5 items()

- Returns a list of (key, value) pairs as tuples:

```
my_dict = {'name': 'Alice', 'age': 25}  
print(my_dict.items())
```

```
# Output: dict_items([('name', 'Alice'), ('age', 25)])
```

4.6 keys()

- Returns a list of keys:

```
print(my_dict.keys())
```

```
# Output: dict_keys(['name', 'age'])
```

5.7 values()

- Returns a list of values:

```
print(my_dict.values())
```

```
# Output: dict_values(['Alice', 25])
```

Note: If you call `items()` and `values()` without changing the dictionary, the order of values will correspond to keys in `items()`.

Functions in Python

1. Introduction to Functions

A **function** in Python is a **named block of code** designed to perform a specific task. Functions allow us to **break a program into smaller, manageable parts**, making it easier to read, debug, and reuse code.

- Functions contain **lines of code** that are executed **sequentially from top to bottom**.
- Using functions reduces code repetition and helps structure programs logically.
- Every Python program can use **built-in functions**, create **user-defined functions**, or use **functions from modules**.

2. Categories of Functions

Functions in Python can be broadly classified into three categories:

i. Module Functions

- A **module** is a separate file that contains Python code such as **functions, classes, and variables**.
- Python provides a **standard library** of modules that contain useful functions for tasks like mathematics, file handling, and string manipulation.
- To use a module, it must be **imported** into the program using the import keyword.

Syntax to import a module:

```
import module_name
```

Example: Using the math module

```
import math

value = math.sqrt(25) # sqrt() function calculates the square root
print(value)         # Output: 5.0
```

- Here, math is the module, sqrt() is a function inside the module, and the **dot (.) notation** is used to access it.

ii. Built-in Functions

- **Built-in functions** are functions that are always available in Python and do **not require importing any module**.
- Python provides a **small set of built-in functions** in the core language. Most functions are organized in modules to keep the language lightweight.

Examples of built-in functions:

```
range(), round(), len(), type(), id()
```

Example usage:

```
my_list = [10, 20, 30, 40]
print(len(my_list)) # Output: 4
```

```
print(round(3.6))    # Output: 4
```

iii. User-Defined Functions

- Python allows programmers to **define their own functions** to perform specific tasks.
- **User-defined functions** are created using the def keyword.
- A function can have **parameters** (inputs) and may return a **value** or just perform an action.

Syntax:

```
def function_name([parameters]):
```

```
# Body of the function
```

```
statements
```

- **Header:** The first line that defines the function (def function_name(parameters):)
- **Body:** The indented block below the header, containing statements executed when the function is called

Example 1: Simple function

```
def sayHello():
```

```
print("Hello World!")
```

```
sayHello()    # Output: Hello World!
```

Example 2: Function with parameter

```
def check(num):
```

```
if num % 2 == 0:
```

```
print(True)
```

```
else:
```

```
print(False)
```

```
check(29) # Output: False
```

- Here, num is a **parameter** – a placeholder for the value we pass when calling the function.

3. Parameters and Arguments

- **Parameters** are variables listed inside the parentheses in a function definition.
- **Arguments** are the actual values passed to the function when calling it.

Example:

```
def area(radius):  
    return 3.14 * radius * radius  
  
print(area(5)) # Argument: 5 → Output: 78.5
```

- The **binding** between parameters and arguments is **1:1**. The number of arguments passed must match the number of parameters unless default values are provided.

4. Default Arguments

- Parameters can have **default values**. If the caller does not provide a value, the function uses the default.
- **Rules:**
 1. Only parameters at the **end of the parameter list** can have default values.
 2. Default values must be **constants**.

Example:

```
def greet(message, times=1):  
    print(message * times)
```

```
greet('Welcome')    # Output: Welcome
greet('Hello', 2)    # Output: HelloHello
```

Another example with multiple defaults:

```
def fun(a, b=1, c=5):
    print('a is', a, 'b is', b, 'c is', c)

fun(3)                # a=3, b=1, c=5
fun(3, 7, 10)         # a=3, b=7, c=10
fun(25, c=20)         # a=25, b=1, c=20
fun(c=20, a=10)       # a=10, b=1, c=20
```

- Functions can be called in **different ways using positional or keyword arguments**, as long as **parameters without default values are provided**.

5. Flow of Execution

- Python executes a program **line by line from top to bottom**.
- **Function definitions do not execute immediately**; they only define the function.
- When a function is **called**, the program **jumps to the function body**, executes all statements, and **returns to the point of the call**.
- If a function calls another function, the program jumps again, executes the called function, and returns to the caller.

Example:

```
def greet():
    print("Hello")

def welcome():
```

```
print("Welcome")  
greet()  
print("Have a nice day!")  
  
welcome()
```

Flow of Execution:

1. welcome() is called
2. Prints "Welcome"
3. Calls greet() → prints "Hello"
4. Returns → prints "Have a nice day!"

Output:

```
Welcome  
  
Hello  
  
Have a nice day!
```

6. Summary

- Functions help **organize code**, **reuse logic**, and **reduce errors**.
- Types of functions:
 - **Module functions** → imported from external modules
 - **Built-in functions** → available in Python by default
 - **User-defined functions** → created by programmers
- **Parameters** allow functions to work on inputs; **arguments** are the actual values.
- **Default values** make some arguments optional.
- **Execution flow** jumps to the function body when called and returns after execution.